

Part 11

Django Forms Handling and Form Validation

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

Django Form Handling

After completing this Django forms tutorial, you will have a great command over these concepts:

- Forms
- HTTP, GET & POST Method
- Django Forms
- Django Form Class
- Django Form Validation

What are Forms.?

Forms are a collection of HTML elements to take input from the user. In simple words, whenever you give any kind of instruction or input on web, that is known as a form.

The input elements come inside **<form></form>** tags. It is collectively called as HTML form. All the websites use forms each having different use-cases.

HTML Forms

The `<form>` tag has two important attributes to be specified:

1. Action

This attribute specifies where to submit the form data. It can be left blank but in that case, the data will be sent to the same URL. Otherwise, you can specify a URL.

2. Method

This attribute specifies how to submit data. There are GET and POST values for the same. You should use capital values for this attribute.

Django Forms Handling

First, we have to understand how the server responds with forms and where do the forms fit in all the client/ server architecture?

1. Render the Form in its Initial State
2. Receive the Form Data & Validate the Information
3. If the Received Data is not Valid, Response is a Bounded Form
4. If the Received Data is Valid, a Success Response will generate

Django Form Class

Forms are implemented via Django form class. Some built-in tasks which Django provides you are:

- **Rendering Form**
- **Form Validation**

Django form class is very similar to the Django models class. Django form class provides you lots of functionality with forms and their validations. This class has many built-in forms which you can use directly to render HTML. You can also define forms, just like models and render the same.

1. Rendering the Form

Creating Forms.py File in the Django Project.

```
from django import forms

#DataFlair #Form
class SignUp(forms.Form):
    first_name = forms.CharField(initial = 'First Name', )
    last_name = forms.CharField()
    email = forms.EmailField(help_text = 'write your email', )
    Address = forms.CharField(required = False, )
    Technology = forms.CharField(initial = 'Django', disabled = True, )
    age = forms.IntegerField()
    password = forms.CharField(widget = forms.PasswordInput)
    re_password = forms.CharField(help_text = 'reenter your password', widget = forms.PasswordInput)
```

Some Other Elements

These fields are going to become different types of input elements when we render them in the HTML file. There are multiple fields for different input elements in [Django](#).

Some types of fields in Django:

- BooleanField
- CharField
- ChoiceField
- DateField
- DecimalField
- EmailField
- FileField, etc.

Attributes of Input Fields

Some common attributes to these fields can be used for more control. These attributes come in handy when validating data. They provide a more constrained selection of data.

- **Required**
- **Initial**
- **Help-text**
- **Disabled**
- **Widget**
- **Validators**
- **Label**

Creating View Function

Now, we need to repeat the process, the same as performed with models. The form will be rendered automatically when passed to HTML. In your views.py add this code:

```
from django.shortcuts import render
from . import forms

#DataFlair #Form #View Functions
def regform(request):
    form = forms.SignUp()
    if request.method == 'POST':
        form = forms.SignUp(request.POST)
        html = 'we have recieved this form again'
    else:
        html = 'welcome for first time'
    return render(request, 'signup.html', {'html': html, 'form': form})
```

Explanation

In above view function, we have declared forms the same as we declared models. We created a new instance of form class. Then we checked by which method we are receiving our form, whether, it's POST method or any other method.

This condition is important. It will check whether the user is requesting the form for the first time or has filled data before. When a user demands form for the first time, we receive a GET request but when a user submits data in form, it will happen through POST request. Thus, we will send the right response.

URL Configuration

Now, we will need to create a new file in the registration directory. *urls.py* should be the name. In that file, add this code:

```
from django.urls import path
from .views import *

#DataFlair #Url-ConfigWhen we validate our form data, we are actually c
urlpatterns = [
    path('', regform, name = 'registration form'),
]
```

Creating HTML templates

Now, we can proceed to render our forms. Create a new folder in your registration directory named templates. In that folder, create a new file named *signup.html*.

```
<h1>Registration Form</h1>

<form action = '' method="post">
  <!-- Very Important csrf Token -->
  {% csrf_token %}
  <table>
    {{ form.as_table }}
  </table>
  <input type="submit" name="register">
</form>
```

Explanation

Here we have a general HTML file. The main difference starts in `<form>` tags. We have two attributes of the form tag: the action which is blank, which means the view function of this page will handle the response and the method, we have specified as the post. If some error arises, try using uppercase POST value.

Then we have used `{% csrf_token %}`. This is the first time we are using any security features.

CSRF stands for Cross-site Request Forgery and is a type of security threat to websites taking form data. Django provides you built-in support for this, by making it necessary to use this token.

Continue...

Without CSRF token, your website forms can be filled by bots. But, your web data is at potential risk of going to other websites.

The CSRF token prevents this situation. This field has some value which is randomly generated by your server and that value shall match with your server when the form is submitted. That particular value is also not acceptable by other servers. Thus, web data remains where it should be.

After that, we used `<table>` tags. Inside that we declared `{{ form.as_table }}`. This is Django's built-in way of form templating. We can declare this as paragraph and more options are available.

Django Forms Validation

Form validation means verification of form data. Form validation is an important process when we are using model forms. When we validate our form data, we are actually converting them to python datatypes. Then we can store them in the database. This validation is necessary since it adds that layer of security where unethical data cannot harm our database.

NOTE:

There are certain JavaScript **Validation** methods too, that can be easier to implement, but they are not preferred at all as JavaScript is client-side scripting. If the client browser does not have JavaScript enabled, the data will not be validated. And, it causes some serious issues for developers.

Validation Of a Particular Field

When we want certain fields to be validated, this method is used. We have to follow some Django conventions at this point to define our methods and other things.

```
#Validation #DataFlair
def clean_password(self):
    password = self.cleaned_data['password']
    if len(password) < 4:
        raise forms.ValidationError("password is too short")
    return password
```

The name of the function is special. You have to define it for each field you want to validate.

Explanation

Syntax:

```
clean_fieldname()
```

Then we used `self.cleaned_data['field_name']`

This is also specific to a particular field. This list contains the cleaned data received from the form. The cleaned data means its in python data-type format. It allows us to use python functions on the data we stored in a variable.

ValidationErrors are built in methods and subclasses of Django. They are used in models too.

Using `is_valid()`

You can use `is_valid()` when required to validate complete form-data. This validation will check for Python-datatypes. This function will return True or False ([Python Data Types](#)) in return.

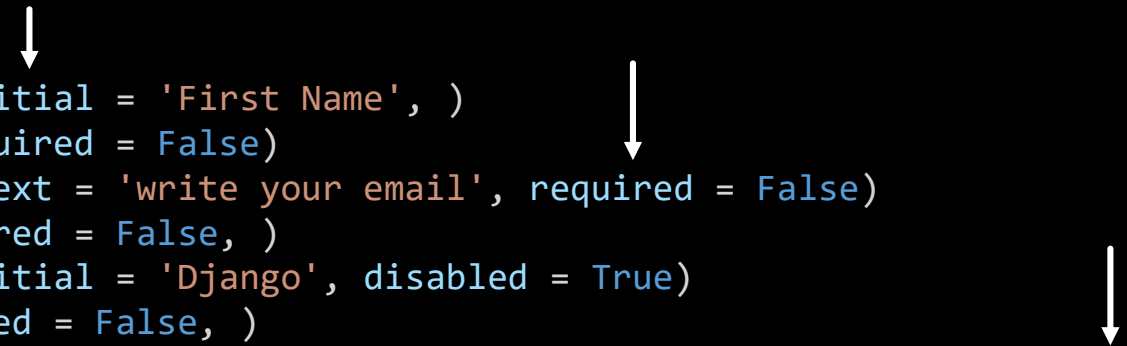
```
def regform(request):
    form = forms.SignUp()
    if request.method == 'POST':
        form = forms.SignUp(request.POST)
        html = 'we have recieved this form again'
        if form.is_valid():
            html = html + "The Form is Valid"
    else:
        html = 'welcome for first time'
    return render(request, 'signup.html', {'html': html, 'form': form})
```

Validation Using Validators

We will now learn a professional way of form validation. It is very easy to use and provide much more control over fields.

```
from django import forms
from django.core import validators

class SignUp(forms.Form):
    first_name = forms.CharField(initial = 'First Name', )
    last_name = forms.CharField(required = False)
    email = forms.EmailField(help_text = 'write your email', required = False)
    Address = forms.CharField(required = False, )
    Technology = forms.CharField(initial = 'Django', disabled = True)
    age = forms.IntegerField(required = False, )
    password = forms.CharField(widget = forms.PasswordInput, validators = [validators.MinLengthValidator(6)])
    re_password = forms.CharField(widget = forms.PasswordInput, required = False)
```



Custom Validators

Suppose you have some fields which you want to validate and there may be no built-in validator of your use-case. Then, you can create your own custom validators.

- 1) Just declare a function with the parameter value. The parameter name shall be that.
- 2) Then, perform your desired validations on value. When value satisfies any criteria raise a `ValidationError`
- 3) Add that `function_name` in `validators` argument of the chosen field, just as we used validators.

Custom Validation

```
from django import forms
from django.core import validators

def check_size(value):
    if len(value) < 6:
        raise forms.ValidationError("the Password is too short")

class SignUp(forms.Form):
    first_name = forms.CharField(initial = 'First Name', )
    last_name = forms.CharField(required = False)
    password = forms.CharField(widget = forms.PasswordInput, validators = [check_size, ])
    re_password = forms.CharField(widget = forms.PasswordInput, required = False)
```

Thanks for Reading